

P1.1 $g_1: X \rightarrow \mathbb{R}$ & $g_2: X \rightarrow \mathbb{R}$ scalar valued func.

to show, $\left| \max_{x \in X} g_1(x) - \max_{x \in X} g_2(x) \right| \leq \max_{x \in X} |g_1(x) - g_2(x)|$

from definition, $|g_1(x) - g_2(x)| \geq g_1(x) - g_2(x) \quad \forall x \in X$

or $g_1(x) \leq |g_1(x) - g_2(x)| + g_2(x) \quad \text{--- (1)}$

now we take max, $(\max_x (A+B) \leq \max_x A + \max_x B) \quad \text{--- (2)}$

on eqn (1) we get
 $\max_{x \in X} g_1(x) \leq \max_{x \in X} g_2(x) + \max_{x \in X} |g_1(x) - g_2(x)|$

Since equality holds as from (2)

i.e., $\max_x (A(x) + B(x)) \leq \max_x A(x) + \max_x B(x)$

property of max operator

$\therefore \max_{x \in X} g_1(x) - \max_{x \in X} g_2(x) \leq \max_{x \in X} |g_1(x) - g_2(x)| \quad \text{--- (3)}$

Same way, $g_2(x) \leq |g_1(x) - g_2(x)| + g_1(x)$ & taking max of.

and so $\max_{x \in X} g_2(x) - \max_{x \in X} g_1(x) \leq \max_{x \in X} (|g_1(x) - g_2(x)|)$
 --- (4)

Combining (3) & (4) we get

$-\max_{x \in X} (|g_1(x) - g_2(x)|) \leq \max_{x \in X} g_1(x) - \max_{x \in X} g_2(x) \leq \max_{x \in X} (|g_1(x) - g_2(x)|)$

$\Rightarrow \max_{x \in X} g_1(x) - \max_{x \in X} g_2(x) \leq \max_{x \in X} |g_1(x) - g_2(x)|$

Q.E.D.

P1.2 To prove $f: X \times Z \rightarrow \mathbb{R}$ & $g: Y \rightarrow Z \subseteq \mathbb{R}$

(2)

$$\max_{x \in X, y \in Y} f(x, g(y)) \geq \max_{x \in X} f(x, \max_{y \in Y} g(y))$$

L.H.S. R.H.S.

RHS first let $y^* \in Y$ be value of y that maximizes $g(y) \therefore g(y^*) = \max_{y \in Y} g(y)$ — (1)

$x^* \in X$ " " " " " " f $\therefore$

$$f(x^*, \max_{y \in Y} g(y)) = \max_{x \in X} f(x, \max_{y \in Y} g(y)) \text{ — (2)}$$

from (1) substitution we get

$$f(x^*, g(y^*)) = \max_{x \in X} f(x, \max_{y \in Y} g(y)) \text{ — (4)}$$

LHS next

hint $h: W \rightarrow \mathbb{R}, \max_{w \in W} h(w) \geq h(w') \quad w' \in W$

LHS is max of $f(x, g(y))$ over $X \times Y$

Using hint, $\max_{x \in X, y \in Y} f(x, g(y)) \geq f(x', g(y'))$
for $x', y' \in X \times Y$

lets choose (x^*, y^*) from above

— (3)

$\therefore$ substitution $\max_{x \in X, y \in Y} f(x, g(y)) \geq f(x^*, g(y^*))$

from (4) $\max_{x \in X, y \in Y} f(x, g(y)) \geq \max_{x \in X} f(x, \max_{y \in Y} g(y))$

Q.E.D.

Problem 2: Consider a Markov decision process (MDP) in the infinite-horizon discounted setting with state space $\mathcal{S} = \{b, c\}$, action space $\mathcal{A} = \{x, y\}$, transition function $P(s'|s, a)$ with

$$\begin{aligned} P(b|b, x) &= 1.0, & P(c|b, x) &= 0.0, \\ P(b|b, y) &= 0.2, & P(c|b, y) &= 0.8, \\ P(b|c, x) &= 0.0, & P(c|c, x) &= 1.0, \\ P(b|c, y) &= 0.6, & P(c|c, y) &= 0.4, \end{aligned}$$

reward function $R(s, a)$ with

$$\begin{aligned} R(b, x) &= 0, & R(b, y) &= 0, \\ R(c, x) &= 1, & R(c, y) &= 1, \end{aligned}$$

and discount factor γ .

1. Suppose we are running a value iteration algorithm over the known MDP model to find the optimal value function. If at iteration $t = 6$, the value function is

$$V_6(b) = 10, \quad V_6(c) = 5,$$

what will the value function be at iteration $t = 7$? The answer will be in terms of γ .

2. Suppose we are running a policy iteration algorithm over the known MDP model to find an optimal policy. If at iteration $t = 8$, the value function for policy π_8 is

$$V^{\pi_8}(b) = 5, \quad V^{\pi_8}(c) = 15,$$

what will the policy be at iteration $t = 9$?

Answer to P2.1

```
In [20]: # ---- MDP ----
states = ["b", "c"] # State space
actions = ["x", "y"] # Action space

# Transition probabilities
P = {
    ("b", "x"): {"b": 1.0, "c": 0.0},
    ("b", "y"): {"b": 0.2, "c": 0.8},
    ("c", "x"): {"b": 0.0, "c": 1.0},
    ("c", "y"): {"b": 0.6, "c": 0.4},
}

# Rewards
R = {
    ("b", "x"): 0.0, ("b", "y"): 0.0,
    ("c", "x"): 1.0, ("c", "y"): 1.0,
}

# Discount
gamma = .95 # I assumed
```

```
In [21]: # Answer Part 1
# V6 is given
V_6 = {"b": 10.0, "c": 5.0}
# V7 can be evaluated from V_next
V_next = {}

for s in states:
    best = float("-inf")
    for a in actions:
        exp_next = 0.0
        for sp in states:
```

```

    exp_next += P[(s,a)][sp] * V_6[sp]
    q = R[(s,a)] + gamma * exp_next
    # print(q)
    if q > best:
        best = q
    V_next[s] = best

print("V7:", V_next) # {'b': 10*gamma, 'c': 1 + 8*gamma}

```

V7: {'c': 8.6}

P2.1 Answer: Value function at $t = 7$: $V_7(b) = 10\gamma$, $V_7(c) = 1 + 8\gamma$

Answer to P2.2

```

In [22]: # Answer Part2:
V_pi8 = {"b": 5.0, "c": 15.0} # given

pi9 = {} # unknown
for s in states:
    best_a = None
    best_q = float("-inf")
    for a in actions:
        # q(s,a) = R(s,a) + gamma * sum_{s'} P(s'|s,a) * V^{pi8}(s')
        exp_next = 0.0
        for sp in states:
            exp_next += P[(s,a)][sp] * V_pi8[sp]
        q = R[(s,a)] + gamma * exp_next
        if q > best_q:
            best_q, best_a = q, a
    pi9[s] = best_a
print("pi9:", pi9) # results

```

pi9: {'b': 'y', 'c': 'x'}

P2.2 The improved policy at $t = 9$ is $\pi_9(b) = y$ and $\pi_9(c) = x$.

Problem 3: Consider an MDP in the infinite-horizon discounted setting over the grid-world illustrated below, where the goal is to take the agent to the treasure chest while avoiding the lightning.



- The state space contains the cells of the grid-world.
- The agent starts at the bottom left corner.
- The action space is $\{\text{up, down, left, right}\}$.
- The agent can only move to its adjacent cells, i.e., the cells that are above, below, to the left, or to the right of its current cell. If the agent is not at the boundary cells, each action will take the agent to the expected cell with probability 0.85 and to one of the remaining three cells, each with probability 0.05. If the agent is at the boundary, it will remain at its current cell with the sum of probabilities that would have taken it outside the grid-world. The cells with a mountain cannot be accessed, i.e., if adjacent to the mountain, the agent will remain at its current cell with the probability that would have taken it to the mountain. All actions in the cell with the lightning bolt, the one with the treasure chest, and the ones with a mountain will keep the agent in its current cell.
- The agent receives a reward of 0 in every cell for all actions except two cells. If at the cell with a lightning bolt, it will receive a reward of -1 for all actions, and if at the cell with the treasure chest, it will receive a reward of $+1$ for all actions.
- The discount factor is 0.95.

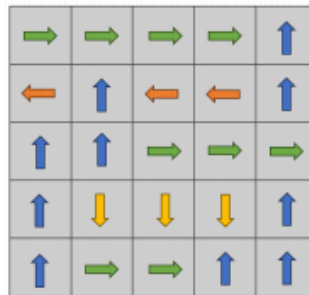
1. Evaluate the deterministic, stationary policy shown in the figure below, where each arrow represents the prescribed action in each cell, over this MDP.
 - (a) Apply the analytical solution for policy evaluation. Report the value function at all states. Also, report the expected discounted cumulative reward (the RL objective) $J(\pi)$ for this policy.
 - (b) Suppose that instead of using the result in Part a, we would like to estimate $J(\pi)$ for this policy using a sampling-based technique. Generate $n = 100$ trajectories of length $T = 50$ according to the MDP and the policy. Compute and report the sample mean of the returns of these trajectories as the estimated RL objective $\hat{J}(\pi)$. Also, compute and report the error in this estimation, that is $|J(\pi) - \hat{J}(\pi)|$.

Purdue University

2

Fall 2025

 ECE 59500: Reinforcement Learning - Theory and Algorithms



- (c) In the sampling-based technique in Part b, how do you expect increasing the length of the sampled trajectories to affect the estimation error? How do you expect increasing the number of the sampled trajectories to affect the estimation error? Explain your reasoning.
 - (d) Implement and run the iterative solution for approximate policy evaluation with a zero initialization for the value function. Pick the number of iterations in a way to ensure 0.01 accuracy in the final computed value function, i.e., $\|V_T - V^\pi\|_\infty \leq 0.01$ without using the results of the previous parts. Justify how you picked the number of iterations. Report the value function at all states.
 - (e) Plot the sequence of errors in the value function from the approximate policy evaluation with respect to the iterations. In particular, plot $\|V_t - V^\pi\|_\infty$ against $t \in \mathbb{N}_0$, where V_t is the value function at iteration t and V^π is the value function computed from the analytical solution.
2. Implement and run a value iteration algorithm to compute an optimal policy in this MDP. Initialize the value function at zero. Pick the number of iterations in a way to ensure 0.01 accuracy in the final computed value function. Demonstrate the learned policy and report the value function at all states.
3. Implement and run a policy iteration algorithm to compute an optimal policy in this MDP. Initialize the policy randomly, by using a uniform distribution over the actions. Pick the number of iterations in a way to ensure 0.01 accuracy in the final computed value function. Demonstrate the learned policy and report the value function at all states.

[Note: For this problem, attach all your code in a programming language of your choice to the end of your submission.]

Answer3.1(a): Analytic policy evaluation


```

In [23]: """
Gridworld policy evaluation

MDP:
- 5x5 grid (x=0 to 4 left→right, y=0 to 4 bottom→top), agent start at (0,0)
- Actions: U,D,L,R. Intended = 0.85, each other = 0.05
- Mountains, Lightning, Treasure are absorbing
- Rewards: -1 on Lightning, +1 on Treasure, 0 elsewhere
- Discount gamma = 0.95

Outputs:
- Value function V_pi (per-state and as a 5x5 grid)
- J(pi) = V_pi(start)
"""

import numpy as np
from typing import Dict, Tuple, List

# Problem spec
W, H = 5, 5
GAMMA = 0.95

# Cell types
N, M, L, T = "N", "M", "L", "T" # normal, mountain, lightning, treasure

# Board Layout (x,y) with (0,0) bottom-left
MOUNTAINS = {(2, 1), (3, 1), (1, 3)}
LIGHTNING = (2, 3)
TREASURE = (4, 4)

# Policy
policy_cols = [
    "URRUU",
    "UDDDU",
    "UURRR",
    "LULLU",
    "RRRRU",
]
policy_grid = np.array([list(col) for col in policy_cols]).T # shape (5,5), ina

# Actions
ACTIONS: Dict[str, Tuple[int, int]] = {
    "U": (0, 1),
    "D": (0, -1),
    "L": (-1, 0),
    "R": (1, 0),
}

DIRECTIONS = ["U", "D", "L", "R"]
P_INTENDED, P_OTHER = 0.85, 0.05

START = (0, 0) # bottom-left

# Helper funcs
def s_idx(x: int, y: int) -> int:
    """(x,y) -> state index"""
    return y * W + x

def inv_idx(s: int) -> Tuple[int, int]:

```

```

"""state index -> (x,y)."""
return (s % W, s // W)

def in_bounds(x: int, y: int) -> bool:
    return 0 <= x < W and 0 <= y < H

def build_board() -> np.ndarray:
    board = np.full((W, H), N, dtype="<U1")
    for (x, y) in MOUNTAINS:
        board[x, y] = M
    board[LIGHTNING] = L
    board[TREASURE] = T
    return board

# ----- Core: build P_pi and r_pi -----
def build_mdp_matrices(policy: np.ndarray, gamma: float = GAMMA):
    board = build_board()
    absorbing = set(MOUNTAINS) | {LIGHTNING, TREASURE}
    nS = W * H

    P = np.zeros((nS, nS), dtype=float) # transition matrix under the policy
    r = np.zeros(nS, dtype=float) # reward vector under the policy

    for x in range(W):
        for y in range(H):
            s = s_idx(x, y)

            # Rewards by state
            if (x, y) == LIGHTNING:
                r[s] = -1.0
            elif (x, y) == TREASURE:
                r[s] = 1.0
            else:
                r[s] = 0.0

            # Mountain, lightning, treasure
            if (x, y) in absorbing:
                P[s, s] = 1.0
                continue

            a = policy[x, y]
            if a not in ACTIONS: # defensive fallback
                P[s, s] = 1.0
                continue

            probs = {d: P_OTHER for d in DIRECTIONS}
            probs[a] = P_INTENDED

            # Distribute probability mass
            for d, p in probs.items():
                dx, dy = ACTIONS[d]
                nx, ny = x + dx, y + dy
                if not in_bounds(nx, ny) or board[nx, ny] == M:
                    P[s, s] += p
                else:
                    P[s, s_idx(nx, ny)] += p

    return P, r, board

# Print funcs

```



```

def print_transitions(P: np.ndarray):
    print("Non-zero transitions under policy (P_pi)")
    nS = P.shape[0]
    for s in range(nS):
        row = P[s]
        nz = np.where(row > 1e-12)[0]
        parts = [f"{s:2d} → {sp:2d}: {row[sp]:.4f}" for sp in nz]
        print(", ".join(parts))
        print()

def print_rewards(r: np.ndarray):
    print("Reward vector r_pi")
    for s, rv in enumerate(r):
        print(f"s={s:2d}: {rv:+.1f}")
    print()

def print_values(v: np.ndarray):
    print("Value function V_pi (per state)")
    for s, vv in enumerate(v):
        print(f"s={s:2d}: {vv: .10f}")
    print("\n V_pi as 5x5 grid (y=top→bottom rows)")
    grid = np.zeros((H, W))
    for s, vv in enumerate(v):
        x, y = inv_idx(s)
        grid[H - 1 - y, x] = vv
    output_lines = []
    for row in grid:
        formatted_row = [f'{x:10.3f}' for x in row]
        output_lines.append(f"[{' '.join(formatted_row)}]")

    print('\n'.join(output_lines))

# Main
def main():
    P, r, board = build_mdp_matrices(policy_grid, GAMMA)
    I = np.eye(P.shape[0])
    A = I - GAMMA * P
    v = np.linalg.solve(A, r)
    # print_transitions(P)
    # print_rewards(r)
    print_values(v)

    start_s = s_idx(*START)
    J = v[start_s]
    print(f"RL objective \nJ(pi) = V_pi(start @ {START}, s={start_s}) = {J:.10f}")

if __name__ == "__main__":
    main()

```

Value function V_{π} (per state)

```
s= 0: 4.3389642925
s= 1: 2.6412756361
s= 2: 2.7064370567
s= 3: 2.8778513233
s= 4: 6.0785827724
s= 5: 4.7074948508
s= 6: 2.6161329734
s= 7: 0.0000000000
s= 8: 0.0000000000
s= 9: 6.6432439272
s=10: 5.1436801551
s=11: 2.8511700123
s=12: 3.7949285003
s=13: 5.4851254748
s=14: 7.0878056624
s=15: 5.6226857792
s=16: 0.0000000000
s=17: -20.0000000000
s=18: -14.2964244507
s=19: 16.5958956510
s=20: 12.0203080131
s=21: 13.1409302505
s=22: 14.0205290973
s=23: 16.9416220165
s=24: 20.0000000000
```

V_{π} as 5x5 grid (y=top→bottom rows)

```
[ 12.020  13.141  14.021  16.942  20.000]
[  5.623   0.000 -20.000 -14.296  16.596]
[  5.144   2.851   3.795   5.485   7.088]
[  4.707   2.616   0.000   0.000   6.643]
[  4.339   2.641   2.706   2.878   6.079]
```

RL objective

$J(\pi) = V_{\pi}(\text{start @ } (0, 0), s=0) = 4.3389642925$

1(b) Answer: sampling-based technique

```
In [24]: # Monte Carlo estimate of J(pi)
n, T = 100, 50
seed = 7
rng = np.random.default_rng(seed)

board = build_board()
absorbing = set(MOUNTAINS) | {LIGHTNING, TREASURE}

returns = []
for _ in range(n):
    x, y = START
    G = 0.0
    disc = 1.0
    for _ in range(T):
        # Reward at current state
        r_t = -1.0 if (x, y) == LIGHTNING else (1.0 if (x, y) == TREASURE else 0.0)

        # Sample one-step dynamics under the policy
        if (x, y) in absorbing:
            nx, ny = x, y
        else:
            a = policy_grid[x, y]
```

```

probs = {d: P_OTHER for d in DIRECTIONS}
probs[a] = P_INTENDED
dirs = np.array(DIRECTIONS)
pvals = np.array([probs[d] for d in dirs])
outcome = rng.choice(dirs, p=pvals)
dx, dy = ACTIONS[outcome]
nx, ny = x + dx, y + dy
# invalid or mountain -> self-loop (stay)
if not in_bounds(nx, ny) or (nx, ny) in MOUNTAINS:
    nx, ny = x, y

# Accumulate discounted return and advance
G += disc * r_t
disc *= GAMMA
x, y = nx, ny

returns.append(G)

returns = np.array(returns, dtype=float)
J_hat = float(returns.mean())

# J_true via analytical eval from 1(a)
P, r_vec, _ = build_mdp_matrices(policy_grid, GAMMA)
v = np.linalg.solve(np.eye(W * H) - GAMMA * P, r_vec)
J_true = float(v[s_idx(*START)])
err = abs(J_true - J_hat)

print("Answer to Part 3(b): Monte Carlo estimate of J(pi)")
print(f"n={n}, T={T}, gamma={GAMMA}, seed={seed}")
print(f"J_true (from part a) = {J_true:.6f}")
print(f"J_hat (MC mean) = {J_hat:.6f}")
print(f"|J - J_hat| = {err:.6f}")

```

Answer to Part 3(b): Monte Carlo estimate of $J(\pi)$

$n=100$, $T=50$, $\gamma=0.95$, $\text{seed}=7$

J_{true} (from part a) = 4.338964

J_{hat} (MC mean) = 2.935459

$|J - J_{\text{hat}}| = 1.403505$

1(c) Answer:

Part (c): **Effect of T and n on the estimation error** $|J(\pi) - \hat{J}(\pi)|$

We know, $|J(\pi) - J_T| = |\mathbb{E} \sum_{t=T}^{\infty} \gamma^t r_t| \leq \sum_{t=T}^{\infty} \gamma^t R_{\max}$

Thus, larger T reduces the truncation contribution to the error in T . Total error drops, then plateaus.

Increasing number of trajectories n on the estimation error $|J(\pi) - \hat{J}(\pi)|$.

From earlier problem $G_i = \sum_{t=0}^{T-1} \gamma^t r_t^{(i)}$ be i.i.d. T -step returns with variance $\sigma_T^2 = \text{var}(G_i)$. The Monte Carlo estimator is $\hat{J}(\pi) = \frac{1}{n} \sum_{i=1}^n G_i$, whose variance satisfies $\text{var}(\hat{J}(\pi)) = \frac{\sigma_T^2}{n} > \text{Std Err}(\hat{J}(\pi)) = \frac{\sigma_T}{\sqrt{n}}$. So, the larger n reduces the sampling contribution to the error at the rate $1/\sqrt{n}$. Reduced standard error.

Answer to 3(d)

```

In [25]: # Part 3(d): Iterative policy evaluation
R_max = 1.0
eps = 0.01 # given
gamma = GAMMA
T_needed = int(np.ceil(np.log(eps * (1 - gamma) / R_max) / np.log(gamma)))
print(f"Using T = {T_needed} iterations (guarantees sup-norm error  $\leq$  {eps}).")

# MDP under the fixed policy
P, r_vec, _ = build_mdp_matrices(policy_grid, GAMMA)

# Iterative evaluation from  $v_0 = 0$ :  $v_{k+1} = r + \gamma P v_k$ 
v = np.zeros(W * H)
for _ in range(T_needed):
    v = r_vec + gamma * (P @ v)

# Report the value function
print_values(v)

# Verification:
v_t = np.linalg.solve(np.eye(W * H) - GAMMA * P, r_vec)
max_error = np.linalg.norm(v_t - v, ord=np.inf)
print(f"Verification:  $\|V_T - V^\pi\|_\infty = \{max\_error:.6e\}")$ 
```

Using T = 149 iterations (guarantees sup-norm error $\leq$ 0.01).
Value function V_π (per state)

```

s= 0:  4.3314138192
s= 1:  2.6335824566
s= 2:  2.6987306232
s= 3:  2.8701417978
s= 4:  6.0708325957
s= 5:  4.6999519072
s= 6:  2.6084828479
s= 7:  0.0000000000
s= 8:  0.0000000000
s= 9:  6.6354909151
s=10:  5.1361432195
s=11:  2.8441057310
s=12:  3.7888980274
s=13:  5.4782365675
s=14:  7.0800521649
s=15:  5.6151207859
s=16:  0.0000000000
s=17: -19.9904093690
s=18: -14.2894788132
s=19:  16.5872720565
s=20:  12.0127060714
s=21:  13.1333261021
s=22:  14.0129248161
s=23:  16.9330062610
s=24:  19.9904093690

```

V_π as 5x5 grid (y=top→bottom rows)

```

[ 12.013  13.133  14.013  16.933  19.990]
[  5.615   0.000 -19.990 -14.289  16.587]
[  5.136   2.844   3.789   5.478   7.080]
[  4.700   2.608   0.000   0.000   6.635]
[  4.331   2.634   2.699   2.870   6.071]

```

Verification: $\|V_T - V^\pi\|_\infty = 9.590631e-03$

Explanation Based on the given $\varepsilon = 0.01$ and $\gamma = 0.95$, the number of iterations T required to guarantee a sup-norm error of at most ε is:

$$T \geq \frac{\log\left(\frac{1}{\varepsilon(1-\gamma)}\right)}{\log(1/\gamma)} = \frac{\log\left(\frac{1}{0.01 \cdot 0.05}\right)}{\log(1/0.95)} \text{ is approx } 148.19$$

So, $T = 149$ iterations was chosen.

The verification step shows that the sup-norm error between the value function after 149 iterations and the true value function (computed analytically) is approximately 9.59×10^{-3} , which is indeed less than the desired $\varepsilon = 0.01$.

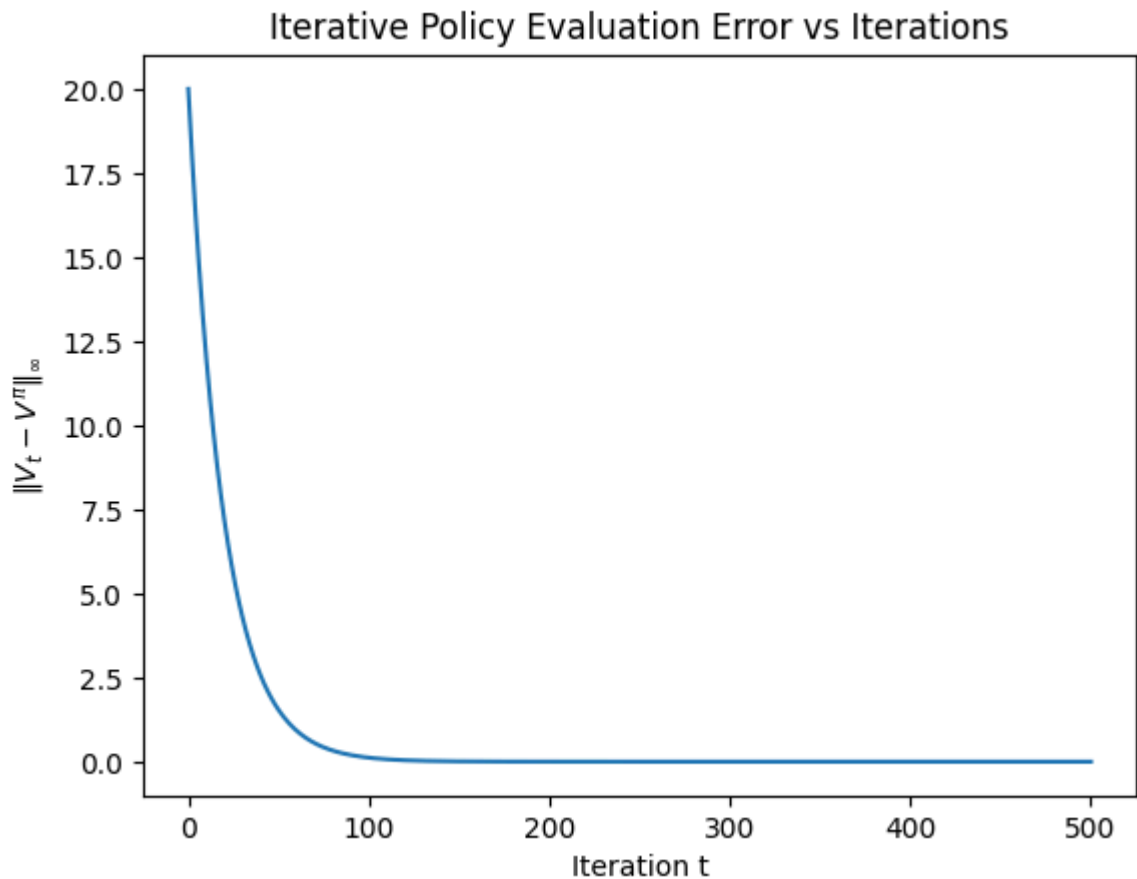
Answer to 3(e)

```
In [26]: import matplotlib.pyplot as plt

# Exact V^pi from analytical solution
I = np.eye(W * H)
v_true = np.linalg.solve(I - GAMMA * P, r_vec)
T_iters = 500
v = np.zeros(W * H, dtype=float)
errors = [np.linalg.norm(v - v_true, ord=np.inf)]

for _ in range(T_iters):
    v = r_vec + GAMMA * (P @ v)
    errors.append(np.linalg.norm(v - v_true, ord=np.inf))

# Plot
plt.figure()
plt.plot(range(len(errors)), errors)
plt.xlabel("Iteration t")
plt.ylabel(r"$\|V_t - V^\pi\|_\infty$")
plt.title("Iterative Policy Evaluation Error vs Iterations")
plt.grid(False)
plt.show()
```



Answer to 3.2

```
In [27]: # Iteration count from contraction bound
eps = 0.01
R_max = 1.0
gamma = GAMMA
T_needed = int(np.ceil(np.log(eps * (1 - gamma) / R_max) / np.log(gamma)))

# Rewards per state
nS = W * H
r_s = np.zeros(nS, dtype=float)
for s in range(nS):
    x, y = s % W, s // W
    r_s[s] = -1.0 if (x, y) == LIGHTNING else (1.0 if (x, y) == TREASURE else 0.

absorbing = set(MOUNTAINS) | {LIGHTNING, TREASURE}

#
def expected_next_value(x, y, a, v_vec):
    if (x, y) in absorbing:
        return v_vec[s_idx(x, y)]
    probs = {d: P_OTHER for d in DIRECTIONS}; probs[a] = P_INTENDED
    ssum = 0.0
    for d in DIRECTIONS:
        dx, dy = ACTIONS[d]
        nx, ny = x + dx, y + dy
        if not in_bounds(nx, ny) or (nx, ny) in MOUNTAINS:
            nx, ny = x, y
        ssum += probs[d] * v_vec[s_idx(nx, ny)]
    return ssum

# value iteration
```

```

v = np.zeros(nS)
for _ in range(T_needed):
    v_new = np.empty_like(v)
    for s in range(nS):
        x, y = s % W, s // W
        v_new[s] = max(r_s[s] + gamma * expected_next_value(x, y, a, v) for a in D
    v = v_new

# greedy policy
opt_policy = {}
for x in range(W):
    for y in range(H):
        if (x, y) in absorbing:
            opt_policy[(x, y)] = "*"
        else:
            opt_policy[(x, y)] = max(DIRECTIONS, key=lambda a: r_s[s_idx(x, y)] +

# outputs
print(f"T_needed = {T_needed} (guarantees  $\|V_T - V^*\|_\infty \leq \{eps\}$ ")
print_values(v)
print("\nOptimal policy (top row first):")
for y in range(H - 1, -1, -1):
    print("".join(opt_policy[(x, y)] for x in range(W)))

```


$T_{\text{needed}} = 149$ (guarantees $\|V_T - V^*\|_{\infty} \leq 0.01$)

Value function V_{π} (per state)

```
s= 0: 11.8498918376
s= 1: 12.6139650108
s= 2: 13.4835636235
s= 3: 14.3702019273
s= 4: 15.3127428056
s= 5: 11.3441779814
s= 6: 11.8827371692
s= 7: 0.0000000000
s= 8: 0.0000000000
s= 9: 16.3169337370
s=10: 11.9857041402
s=11: 12.5084694920
s=12: 13.3510678156
s=13: 16.1891050958
s=14: 17.3869301473
s=15: 12.7353604407
s=16: 0.0000000000
s=17: -19.9904093690
s=18: 15.7151904704
s=19: 18.5975114130
s=20: 13.5685462059
s=21: 14.4582358964
s=22: 15.4063384908
s=23: 18.4987448202
s=24: 19.9904093690
```

V_{π} as 5x5 grid (y=top→bottom rows)

```
[ 13.569  14.458  15.406  18.499  19.990]
[ 12.735   0.000 -19.990  15.715  18.598]
[ 11.986  12.508  13.351  16.189  17.387]
[ 11.344  11.883   0.000   0.000  16.317]
[ 11.850  12.614  13.484  14.370  15.313]
```

Optimal policy (top row first):

```
RRRR*
U**RU
URRRU
UD**U
RRRRU
```

Answer to 3.3

```
In [28]: # 3.3: Policy Iteration
eps = 0.01
A_LETTERS = DIRECTIONS # ["U", "D", "L", "R"]
nA, nS = len(A_LETTERS), W * H

# Per-action transitions P_a
def build_P_for_action(a_letter):
    P_a = np.zeros((nS, nS), dtype=float)
    for x in range(W):
        for y in range(H):
            s = s_idx(x, y)
            if (x, y) in ({LIGHTNING, TREASURE} | set(MOUNTAINS)):
                P_a[s, s] = 1.0
                continue
            probs = {d: P_OTHER for d in DIRECTIONS}; probs[a_letter] = P_INTENDED
            for d, p in probs.items():
```

```

        dx, dy = ACTIONS[d]
        nx, ny = x + dx, y + dy
        if not in_bounds(nx, ny) or (nx, ny) in MOUNTAINS:
            P_a[s, s] += p
        else:
            P_a[s, s_idx(nx, ny)] += p
    return P_a

P_actions = {a: build_P_for_action(a) for a in A_LETTERS}

# State-only rewards
r_vec = np.zeros(nS, dtype=float)
for s in range(nS):
    x, y = s % W, s // W
    if (x, y) == LIGHTNING: r_vec[s] = -1.0
    elif (x, y) == TREASURE: r_vec[s] = 1.0

# Uniform random initial policy
pi_probs = np.full((nS, nA), 1.0 / nA, dtype=float)
prev_a_idx = None

while True:
    # Policy evaluation:  $v = (I - \gamma P^{\pi})^{-1} r$ 
    P_pi = sum((pi_probs[:, i][:, None]) * P_actions[a] for i, a in enumerate(A_LETTERS))
    v = np.linalg.solve(np.eye(nS) - GAMMA * P_pi, r_vec)

    # Greedy improvement (deterministic: single argmax per state)
    Q = np.stack([r_vec + GAMMA * (P_actions[a] @ v) for a in A_LETTERS], axis=1)
    a_idx = Q.argmax(axis=1)

    # Optimality residual for accuracy:  $\|v - T^* v\|_{\infty} \leq (1-\gamma) * \epsilon \Rightarrow \text{error} \leq \epsilon$ 
    residual = np.max(np.abs(v - Q.max(axis=1)))
    if prev_a_idx is not None and np.array_equal(a_idx, prev_a_idx) and residual < epsilon:
        break

    # Make new policy
    pi_probs = np.zeros_like(pi_probs)
    pi_probs[np.arange(nS), a_idx] = 1.0
    prev_a_idx = a_idx

# Report learned policy and values
print("Learned policy (top row first):")
pi_letters = np.array(A_LETTERS)[a_idx].reshape(H, W)
for row in pi_letters[::-1]:
    print("".join(row))

print_values(v)
print(f"\nFinal optimality residual = {residual:.6e}  $\Rightarrow$  error bound  $\leq$  {residual:.6e}")

```

Learned policy (top row first):

RRRRU

UUURU

URRRU

UDUUU

RRRRU

Value function V_{π} (per state)

s= 0: 11.8593535836

s= 1: 12.6234816316

s= 2: 13.4930899234

s= 3: 14.3797287965

s= 4: 15.3222697083

s= 5: 11.3527068554

s= 6: 11.8921441203

s= 7: 0.0000000000

s= 8: 0.0000000000

s= 9: 16.3264606417

s=10: 11.9941264877

s=11: 12.5168901366

s=12: 13.3594303421

s=13: 16.1985202717

s=14: 17.3964570522

s=15: 12.7437766222

s=16: 0.0000000000

s=17: -20.0000000000

s=18: 15.7237589031

s=19: 18.6070448902

s=20: 13.5769620247

s=21: 14.4666516939

s=22: 15.4147542870

s=23: 18.5082198179

s=24: 20.0000000000

V_{π} as 5x5 grid (y=top→bottom rows)

[13.577 14.467 15.415 18.508 20.000]

[12.744 0.000 -20.000 15.724 18.607]

[11.994 12.517 13.359 16.199 17.396]

[11.353 11.892 0.000 0.000 16.326]

[11.859 12.623 13.493 14.380 15.322]

Final optimality residual = $3.552714e-15 \Rightarrow$ error bound $\leq 7.105427e-14$ (target 0.01)

P2.1 See code 1 MKD for answer
 P2.2 " " 2 MKD for answer.

P3. See code 2 MKD for answers. :- [why so long!]

P4.1 MDP $(S, \mu_0, A, P, R, \gamma)$ in the infinite horizon discounted setting

Stationary state occupancy measure $\nu_{\mu_0}^{\pi}(s, a)$ under stationary stochastic policy, π :

$$\nu_{\mu_0}^{\pi}(s, a) = \sum_{t=0}^{\infty} \gamma^t \mathbb{P}(S_t = s, A_t = a | \mu_0, \pi, P) \quad \text{--- (1)}$$

To prove $\forall s \in S$

$$\sum_{a \in A} \nu_{\mu_0}^{\pi}(s, a) = \mu_0(s) + \gamma \sum_{s' \in S} \sum_{a' \in A} P(s | s', a') \nu_{\mu_0}^{\pi}(s', a')$$

LHS

$$\sum_{a \in A} \nu_{\mu_0}^{\pi}(s, a) = \sum_{a \in A} \sum_{t=0}^{\infty} \gamma^t \mathbb{P}(S_t = s, A_t = a | \mu_0, \pi, P) \quad \text{from defn --- (1)}$$

$$= \sum_{t=0}^{\infty} \gamma^t \sum_{a \in A} \mathbb{P}(S_t = s, A_t = a | \mu_0, \pi, P) \quad \because \gamma < 1 \quad \text{--- (2)}$$

in (2) RHS

$$\sum_{a \in A} \mathbb{P}(S_t = s, A_t = a | \mu_0, \pi, P) = \mathbb{P}(S_t = s | \mu_0, \pi, P) = \mu_t^{\pi}(s) \quad \text{(in state s at t) let}$$

(2) becomes

$$\therefore \sum_{a \in A} \nu_{\mu_0}^{\pi}(s, a) = \sum_{t=0}^{\infty} \gamma^t \mu_t^{\pi}(s) \quad \text{by substn}$$

$$= \gamma^0 \mu_0^{\pi}(s) + \sum_{t=1}^{\infty} \gamma^t \mu_t^{\pi}(s)$$

or

$$\sum_{a \in A} \nu_{\mu_0}^{\pi}(s, a) = \mu_0(s) + \sum_{t=1}^{\infty} \gamma^t \mu_t^{\pi}(s) \quad \text{--- (3)}$$

(4)

For $t \geq 1$

$$\mu_t^\pi(s) = \mathbb{P}(S_t = s) = \sum_{s' \in S} \sum_{a' \in A} \mathbb{P}(S_t = s | S_{t-1} = s', A_{t-1} = a') \mathbb{P}(S_{t-1} = s', A_{t-1} = a')$$

Some substitution

$$= \sum_{s' \in S} \sum_{a' \in A} P(s | s', a') \mathbb{P}(S_{t-1} = s', A_{t-1} = a' | \mu_0, \pi, P) \quad \text{--- (4)}$$

substⁿ of (4) in (3) r.h.s (second term)

$$\begin{aligned} \sum_{t=1}^{\infty} \gamma^t \mu_t^\pi(s) &= \sum_{t=1}^{\infty} \gamma^t \left(\sum_{s' \in S} \sum_{a' \in A} P(s | s', a') \mathbb{P}(S_{t-1} = s', A_{t-1} = a' | \mu_0, \pi, P) \right) \\ &= \gamma \sum_{t=1}^{\infty} \gamma^{t-1} \left(\sum_{s' \in S} \sum_{a' \in A} P(s | s', a') \mathbb{P}(S_{t-1} = s', A_{t-1} = a' | \mu_0, \pi, P) \right) \\ &= \gamma \sum_{k=0}^{\infty} \gamma^k \left(\sum_{s' \in S} \sum_{a' \in A} P(s | s', a') \mathbb{P}(S_k = s', A_k = a' | \mu_0, \pi, P) \right) \end{aligned}$$

t-1 = k (say)

$$\therefore \sum_{t=1}^{\infty} \gamma^t \mu_t^\pi(s) = \gamma \sum_{s' \in S} \sum_{a' \in A} P(s | s', a') v_{\mu_0}^\pi(s', a') \quad \text{--- (5)}$$

 $\therefore$ substⁿ of (5) in eqⁿ (3)

$$\sum_{a \in A} v_{\mu_0}^\pi(s, a) = \mu_0(s) + \gamma \sum_{s' \in S} \sum_{a' \in A} P(s | s', a') v_{\mu_0}^\pi(s', a') \quad \text{Q.E.D}$$

--- x ---

P4.2 State occupancy measure $p_{\mu_0}^\pi(s)$ under stationary stochastic policy π as

$$p_{\mu_0}^\pi(s) = \sum_{t=0}^{\infty} \gamma^t \mathbb{P}(S_t = s | \mu_0, \pi, P) \quad \text{--- (A) oh. I love writing! :-}$$

To prove $p_{\mu_0}^\pi(s) = \sum_{a \in A} v_{\mu_0}^\pi(s, a) \quad \text{--- (1)}$

$$v_{\mu_0}^\pi(s, a) = p_{\mu_0}^\pi(s) \pi(a | s) \quad \text{--- (2)}$$

To prove ①,

$$\begin{aligned} \text{RHS } \sum_{a \in A} V_{\mu_0}^{\pi}(s, a) &= \sum_{a \in A} \left(\sum_{t=0}^{\infty} \gamma^t \mathbb{P}(S_t = s, A_t = a | \mu_0, \pi, P) \right) \\ &= \sum_{t=0}^{\infty} \gamma^t \left(\sum_{a \in A} \mathbb{P}(S_t = s, A_t = a | \mu_0, \pi, P) \right) \\ &= \gamma^t \mathbb{P}(S_t = s | \mu_0, \pi, P) = p_{\mu_0}^{\pi}(s) \end{aligned}$$

joint prob

$$\therefore p_{\mu_0}^{\pi}(s) = \sum_{a \in A} V_{\mu_0}^{\pi}(s, a) \quad \text{Q.E.D.}$$

To prove ②,

$$V_{\mu_0}^{\pi}(s, a) = \sum_{t=0}^{\infty} \gamma^t \mathbb{P}(S_t = s, A_t = a | \mu_0, \pi, P)$$

↓
 $\mathbb{P}(A_t = a | S_t = s) \mathbb{P}(S_t = s)$
 $\pi(a|s) \mathbb{P}(S_t = s | \mu_0, \pi, P)$
 π policy stationary

$$\begin{aligned} \therefore V_{\mu_0}^{\pi}(s, a) &= \sum_{t=0}^{\infty} \gamma^t (\pi(a|s) \mathbb{P}(S_t = s | \mu_0, \pi, P)) \\ &= \pi(a|s) \sum_{t=0}^{\infty} \gamma^t \mathbb{P}(S_t = s | \mu_0, \pi, P) \\ &= \pi(a|s) \cdot p_{\mu_0}^{\pi}(s) \end{aligned}$$

that's r.h.s of eqn ①

$$V_{\mu_0}^{\pi}(s, a) = p_{\mu_0}^{\pi}(s) \cdot \pi(a|s) \quad \text{Q.E.D.}$$

4.3 4.1 $0 \leq \gamma < 1$ as $p_{\mu_0}^{\pi} = \mu_0 + \gamma P^{\pi T} p_{\mu_0}^{\pi}$

$$\sum_{a \in A} V_{\mu_0}^{\pi}(s, a) = \sum_{t=0}^{\infty} \gamma^t \mathbb{P}(S_t = s, A_t = a | \mu_0, \pi, P) \quad \text{--- ①}$$

$$\sum_a V_{\mu_0}^{\pi}(s, a) = p_{\mu_0}^{\pi}(s) \quad \text{--- ②}$$

earlier proved, that's like the LHS

$p_{\mu_0}^{\pi} \in \mathbb{R}^{|S|}$
 col vector of
 $p_{\mu_0}^{\pi}(s) \forall s \in S$
 $\mu_0 \in \mathbb{R}^{|S|}$
 $P^{\pi} \in \mathbb{R}^{|S| \times |A|}$
 prob tran matrix for ind MC π

also $\vartheta_{\mu_0}^{\pi}(s, a) = p_{\mu_0}^{\pi}(s) \pi(a|s)$ also proved earlier.

Sub: RHS of eqn ① for $\vartheta_{\mu_0}^{\pi}$ from above

$$\gamma \sum_{s' \in S} \sum_{a' \in A} P(s|s', a') \vartheta_{\mu_0}^{\pi}(s', a') = \gamma \sum_{s' \in S} \sum_{a' \in A} P(s|s', a') [p_{\mu_0}^{\pi}(s') \pi(a'|s')] \\ = \gamma \sum_{s' \in S} p_{\mu_0}^{\pi}(s') \left[\sum_{a' \in A} \pi(a'|s') P(s|s', a') \right] \\ \underbrace{\sum_{a' \in A} \pi(a'|s') P(s|s', a')}_{P^{\pi}(s|s')}$$

$$= \gamma \sum_{s' \in S} p_{\mu_0}^{\pi}(s') P^{\pi}(s|s') \quad \text{--- ③}$$

$\therefore$ from ② & ③,

$$p_{\mu_0}^{\pi}(s) = \mu_0(s) + \gamma \sum_{s' \in S} p_{\mu_0}^{\pi}(s') P^{\pi}(s|s') \quad \text{--- ④}$$

By def: $P^{\pi}(s|s') = \begin{bmatrix} P^{\pi}(s_1|s_1) & \dots & P^{\pi}(s_1|s_n) \\ P^{\pi}(s_n|s_1) & \dots & P^{\pi}(s_n|s_n) \end{bmatrix}_{|S| \times |S|}$

$\rightarrow$ Curr $s' \rightarrow$ Col
 $\rightarrow$ next $s \rightarrow$ row

$$p_{\mu_0}^{\pi} = \begin{bmatrix} p_{\mu_0}^{\pi}(s_1) \\ \vdots \\ p_{\mu_0}^{\pi}(s_n) \end{bmatrix}$$

$$\mu_0 = \begin{bmatrix} \mu_0(s_1) \\ \vdots \\ \mu_0(s_n) \end{bmatrix}$$

$\therefore$ $\left[\begin{matrix} P^{\pi} \\ \mu_0 \end{matrix} \right] =$ eqn ④ can be written as

$$\begin{aligned} P(s_1) &= \mu_0(s_1) + \gamma [P^{\pi}(s_1|s_1)P(s_1) + P^{\pi}(s_1|s_2)P(s_2) + \dots + P^{\pi}(s_1|s_n)P(s_n)] \\ \vdots \\ P(s_n) &= \mu_0(s_n) + \gamma [P^{\pi}(s_n|s_1)P(s_1) + \dots + P^{\pi}(s_n|s_n)P(s_n)] \end{aligned}$$

So to vectorize so that we can do the multiplication with $P^{\pi}_{|S| \times |S|}$ which is a colⁿ vector with P^{π} which is $|S| \times |S|$ in vector form it will become

$$p_{\mu_0}^{\pi} = \mu_0 + \gamma (P^{\pi})^T p_{\mu_0}^{\pi} \quad \xrightarrow{\text{Transpose coz } s' \rightarrow s} \boxed{\text{Answer}}$$

[Since $[P^{\pi T} p_{\mu_0}^{\pi}] = \sum_{s' \in S} [P^{\pi}]_{s, s'} [p_{\mu_0}^{\pi}]_{s'} = \sum_{s' \in S} [P^{\pi}]_{s, s'} p_{\mu_0}^{\pi}(s')$ makes r.h.s of eqn ④

$$= \sum P^{\pi}(s|s') p_{\mu_0}^{\pi}(s') \\ = \sum p_{\mu_0}^{\pi}(s') P^{\pi}(s|s')]$$

4.4 Part 3 derivation $\rho_{\mu_0}^\pi = \mu_0 + \gamma P^{\pi T} \rho_{\mu_0}^\pi$ — (1) (7)

or $\rho_{\mu_0}^\pi - \gamma P^{\pi T} \rho_{\mu_0}^\pi = \mu_0$

or $(I - \gamma P^{\pi T}) \rho_{\mu_0}^\pi = \mu_0$

or $\rho_{\mu_0}^\pi = (I - \gamma P^{\pi T})^{-1} \mu_0$ — (2)

This form (2) is an analytical form similar to HW2 prob 5. to show uniqueness as before $(I - \gamma P^{\pi T})$ is invertible.

if $I - \gamma P^{\pi T}$ is invertible, $I - \gamma P^{\pi T}$ is also invertible

so we try to find if any eigenvalue is equal to zero.

since $\lambda_i(P^{\pi T})$ an eigenvalue of $P^{\pi T}$ so, $\lambda_i(P^{\pi T}) \leq 1$ & $\lambda_i(P^{\pi T}) = \lambda_i(P^\pi)$

if we assume $(I - \gamma P^{\pi T})$ is singular $\forall \bar{x} \neq 0$ s.t. $(I - \gamma P^{\pi T})\bar{x} = 0$

$\therefore \bar{x} - \gamma P^{\pi T} \bar{x} = 0$ or $\gamma P^{\pi T} \bar{x} = \bar{x}$ or $P^{\pi T} \bar{x} = \frac{1}{\gamma} \bar{x}$ like [HW2 Soln]

so, $\frac{1}{\gamma}$ is an eigenvalue of $P^{\pi T}$ say λ_i since largest eigenvalue is 1 for $P^{\pi T}$ and $0 < \gamma < 1$ so $\frac{1}{\gamma} > 1$ i.e. $\lambda_i > 1$

hence $(I - \gamma P^{\pi T})$ is invertible & since system of linear eqns have unique solution iff A matrix is invertible that guarantees uniqueness. Answer

4.5 From 4.2 & 4.3 & 4.2

$v_{\mu_0}^\pi(s, a) = \rho_{\mu_0}^\pi(s) \pi(a|s)$

$$\pi(a|s) = \frac{v_{\mu_0}^\pi(s, a)}{\rho_{\mu_0}^\pi(s)} = \frac{v_{\mu_0}^\pi(s, a)}{\sum_{a' \in \mathcal{A}} v_{\mu_0}^\pi(s, a') \rightarrow \neq 0}$$

Stationary & stochastic policy comply

since $\pi(a|s)$ has no time step t , it is stationary.

also stochastic because denominator $\sum_{a'} v_{\mu_0}^\pi(s, a') \neq 0$ &

numerator $v_{\mu_0}^\pi(s, a) \geq 0$ are non negative for dual LP

2 for any state $\sum v_{\mu_0}(s, a) > 0$ $0 \leq \pi(a|s) \leq 1$ 2 $\sum_a \pi(a|s) = 1$ (8)

$\therefore \pi$ is well defined stationary & stochastic.

from 4.3 & 4.4 we have $P_{\mu_0}^{\pi} = \mu_0 + \gamma (P^{\pi})^T P_{\mu_0}^{\pi}$, so its a relationship b/w policy & probability transitions. and state occupancy is in this eqn.

Also if say $v_{\mu_0}^{(1)}$ is optimal solution to the dual LP then the policy defined by $\pi^{(1)}(a|s) = \frac{v_{\mu_0}^{(1)}(s, a)}{\sum_{a' \in A} v_{\mu_0}^{(1)}(s, a')}$ must have same measure.

so, an optimal, stationary, stochastic policy should comply.

Ex. 6. Given
$$v^{\pi}(\mu_0) = \mathbb{E}_{\substack{S_0 \sim \mu_0 \\ A_t \sim \pi(\cdot|S_t) \\ S_{t+1} \sim P(\cdot|S_t, A_t)}} \left[\sum_{t=0}^{\infty} \gamma^t R(S_t, A_t) \right] \quad \text{--- (1)}$$

$$v^{\pi}(\mu) = \mathbb{E}_{S \sim \mu} [v^{\pi}(S)] = \sum_{s \in S} \mu(s) v^{\pi}(s)$$

To prove $J(\pi) = v^{\pi}(\mu_0) = \sum_{s \in S} \sum_{a \in A} v_{\mu_0}^{\pi}(s, a) R(s, a)$

$$v^{\pi}(\mu_0) = \mathbb{E}_{\substack{S_0 \\ A_t \\ S_{t+1}}} \left[\sum_{t=0}^{\infty} \gamma^t R(S_t, A_t) \right] \quad \text{from (1)}$$

$$= \sum_{t=0}^{\infty} \gamma^t \mathbb{E} [R(S_t, A_t)]$$

$$= \sum_{t=0}^{\infty} \gamma^t \left(\sum_{s \in S} \sum_{a \in A} P(S_t = s, A_t = a | \mu_0, \pi) R(s, a) \right)$$

$$= \sum_{s \in S} \sum_{a \in A} R(s, a) \left(\sum_{t=0}^{\infty} \gamma^t P(S_t = s, A_t = a | \mu_0, \pi) \right)$$

$v_{\mu_0}^{\pi}(s, a)$ by defn

$$J(\pi) = v^{\pi}(\mu_0) = \sum_{s \in S} \sum_{a \in A} v_{\mu_0}^{\pi}(s, a) \cdot R(s, a) \quad \text{Q.E.D}$$

PA.7

Objective

$$\max_{\pi} \sum_{s,a} v(s,a) R(s,a)$$

state occupancy measure

$$J(\pi) = \sum_{s,a} v_{\pi}(s,a) R(s,a)$$

By maximizing the objective function seeks to maximize the total expected reward accumulated across all states / actions. By increasing expected discounted reward it finds the occupancy measure that has highest value. So the objective is maximized the value of the policy by the occupancy, v .

$$\sum_{a \in A} v(s,a) = \mu_0(s) + \gamma \sum_{\substack{s',a' \in \mathcal{S} \\ s' \in \mathcal{S}}} P(s|s',a') v(s',a') \quad \forall s \in \mathcal{S}$$

discounted flow of prob out of state s

start at $t=0$

dynamics

total discounted flow from previous state (s',a') to states s

the constraint link the abstract measure v to the MDP dynamics.

Once dual LP is solved & we get optimal occupancy measure $v^*(s,a)$ & policy π^* which is also optimal

$$\text{so } \pi^*(a|s) = \frac{v^*(s,a)}{\sum_{a' \in A} v^*(s,a')} \quad \forall s \quad \& \quad \sum_{a' \in A} v^*(s,a') \neq 0$$

Answer

— x —